오토인코더의 성능 최적화를 위한

하이퍼파라미터 튜닝

Hyperparameter Tuning for Optimizing Autoencoder Performance

숭실대학교 AI융합학부 20203076 윤태근

숭실대학교 AI융합학부 20203163 홍형준

요 약

본 연구는 오토인코더 모델의 성능을 최적화하기 위한 하이퍼파라미터 튜닝 과정을 탐구한다. 오토인코더는 비지도 학습에서 차원 축소와 이상 탐지에 널리 사용되는 모델로, 본 연구에서는 레이어 수, 노드 수, 학습률, 에포크 수, 배치 크기, 옵티마이저, 임계값 설정 등 주요 하이퍼파라미터를 조정하여 성능에 미치는 영향을 분석한다. Recall, F1-score, Loss 등 성능 지표를 최적화하여 최적의 재구성 성능을 도출하는 것이 목표이고 오토인코더의 하이퍼파라미터 조합에 따른 성능 향상에 대한 방향성을 제공한다.

ABSTRACT

This study explores the process of hyperparameter tuning to optimize the performance of an Autoencoder model. Autoencoders are widely used in unsupervised learning for dimensionality reduction and anomaly detection. In this research, we analyze the impact of key hyperparameters, including the number of layers, nodes, learning rate, epoch count, batch size, optimizer, and threshold settings, on model performance. The goal is to optimize performance metrics such as Recall, F1-score, and Loss to achieve the best reconstruction performance. Additionally, this study provides insights into performance improvements based on various combinations of Autoencoder hyperparameters.

키워드 : 오토인코더. 하이퍼파라미터 튜닝, 비지도 학습, 이상 탐지

Key Words : Autoencoder, Hyperparameter Tuning, Unsupervised Learning, Anomaly Detection

---

2024년 2학기 고급인공신경망

Ⅰ. 서 론

최근 오토인코더(Autoencoder)는 이미지, 음성, 텍스트 등 다양한 분야에서 성공적으로 활용되며 특히, 비선형적인 데이터의 특징을 추출하는 데 뛰어난 성능을 보인다. 본 보고서에서는 오토인코더의 최적 성능 평가 지표를 얻기 위한 하이퍼파라미터 튜닝 과정에 대해 서술하고, 오토인코더의 활용 분야에 대해 탐구한다.

오토인코더는 입력 데이터를 압축하고, 이를 다시 복원하는 과정에서 입력과 유사한 출력을 생성하도록 학습되는 인공신경망이다. 주로 입력 데이터의 특성을 잘 유지하며 저차원 잠재공간(Latent Space)으로 데이터를 표현하는 데 사용된다. 오토인코더는 입력과 출력의 차이를 최소화하는 방식으로 학습되며, 그 결과 데이터의 중요한 패턴을 추출한다.

비지도학습(Unsupervised learning)은 데이터에 대한 명확한 정답 레이블이 없는 상황에서 유용한 패턴이나 구조를 찾는 학습 방식이다. 오토인코더를 사용하는 목적은 차원 축소, 이상치 탐지, 데이터 압축 등이 있다. 특히 데이터가 고차원일 경우, 유의미한 잠재 변수를 통해 원본 데이터의 핵심 정보를 보존하면서 차원을 축소하는 것이 가능하다. 오토인코더는 단순한 데이터 재구성 모델을 넘어, 비지도학습 기반의 딥러닝 모델로 점점 더 많은 주목을 받고 있다. 기존의 차원 축소 기법(PCA 등)과 비교하여, 비선형적 관계를 학습할 수 있는 강점을 가지며, 이상치 탐지나 이미지 복원, 데이터 생성 등의 다양한 응용 분야에서 활용되고 있다.

원본 오토인코더 모델에서 더 나은 오토인코더 모델로 발전하기 위해서는 주요 성능 지표의 값이 향상되어야 한다. 이러한 성능 향상을 이루기 위해서는 모델 구조와 다양한 하이퍼파라미터 조합을 적절하게 구성해야 한다. 원본 오토인코더의 구조는 [그림 1]과 같다. 중앙을 기준으로 앞부분은 Encoder, 뒷부분은 Decoder로 구성되어 있으며, 모래시계 형태의 구조를 갖추고 있다. Encoder는 입력 데이터를 더 작은 차원으로 압축하는 신경망이다. 인코더는 데이터의 주요 특징을 추출하여 잠재공간에 표현한다. Decoder는 잠재공간에 압축된 데이터를 원래의 차원으로 복원하는 신경망이다. Encoder와 Decoder의 구성 요소로는 Linear Layer와 Activation Function(활성화 함수)가 있다.

본 보고서의 목적은 모델 구조, 오토인코더의 하이퍼파라미터, 로스 함수 및 활성화 함수를 조정하여 학습 성능을 최적화하는 방법에 대해 탐구하고자 한다. 하이퍼파라미터 조합을 통해 원본 오토인코더 모델에서 더 나은 모델로 발전하기 위한 방안을 제시하며, 오토인코더 모델의 구성과 학습 구조에 대해 다룬다.

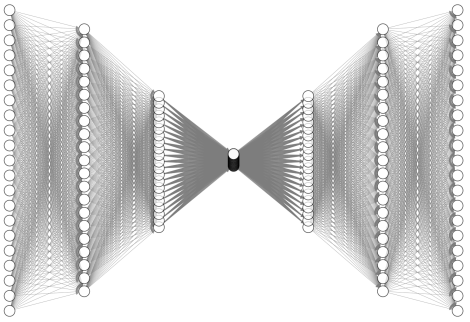

그림 1. 오토인코더 모델 구조
Fig. 1. Autoencoder model architecture

Ⅱ. 본 론

본 장에서는 모델 구조, 정규화, 하이퍼파라미터, 로스 함수, 활성화 함수를 조정하여 오토인코더의

성능을 향상시키는 과정을 제시한다.

2.1 **모델 구조**

모델의 구조는 오토인코더의 성능에 큰 영향을 미친다. 본 프로젝트의 조건은 Latent_size는 32, 인코더와 디코더는 대칭이 되도록하고 이 조건에 맞게 레이어 수와 각 레이어의 히든 노드 수를 변화시켜 적합한 모델의 구조를 찾았다.

2.1.1 **레이어 수**

더 많은 레이어는 더 복잡한 데이터 패턴을 학습할 수 있다. 따라서 복잡한 데이터나 다양한 특성이 존재하는 데이터셋의 경우, 레이어 수를 늘리면 더 나은 정확도를 얻을 가능성이 있다. 하지만, 레이어가 너무 많으면 과적합 위험이 있다. 또한 과도하게 많은 레이어를 추가하면 학습 시간이 증가하고, 기울기 소실 문제가 발생한다.

2.1.2 **레이어 노드 수**

레이어마다 노드 수를 조절하여 모델의 복잡성을 제어할 수 있다.

2.1.3 **정규화**

정규화는 모델의 학습 속도와 안정성을 높이고 과적합을 방지한다. 정규화 방법에는 Batch Normalization, Layer Normalization 등이 있다.

2.1.4 **활성화 함수**

활성화 함수는 각 레이어의 출력값을 결정하는 비선형 변환 함수이다. 활성화 함수의 종류에는 Sigmoid, Tanh, ReLU, LeakyReLU, ELU 등이 있으며, 데이터셋의 특성에 따라 적절한 함수를 선택하되, 마지막 활성화 함수는 Sigmoid로 고정하여 실험하였다.

2.2 Hyper parameter

2.2.1 Batch size

Batch size는 학습할 때 한 번의 학습 단계에서 사용할 샘플 데이터 수이다. Batch size는 학습 속도와 성능에 영향을 준다. Batch size는 일반적으로 2의 지수 형태로 설정하며 사용하고 있는 메모리에 따라 결정된다. 본 보고서는 Google Colab T4 환경에서 적절한 Batch size를 찾았다.

2.2.2 Learning rate

학습률은 경사 하강법 중 가중치를 갱신 하는 비율을 나타낸다. 너무 큰 학습률은 수렴하는 최적점에 도달하지 못 할 수있고 너무 작은 학습률은 도달하는 시간이 너무 오래걸릴 수 있다. 학습률은 Batch size랑 연관이 크기 때문에 함께 조절하여 선택한다.

2.2.3 Epoch 수

에포크 수는 모든 과정 후 마지막에 결정하였다. 에포크는 학습 데이터 전체를 모델이 몇 번 반복하여 학습할 것인지를 결정하는 하이퍼파라미터다. 너무 작으면 과소적합이 너무 크면 과적합이 생기는 위험이 있다. Batch size와 학습률과 연관이 깊기에 적절한 에포크 수를 찾아 성능을 높여야 한다.

2.3 **로스 함수**

로스 함수는 모델이 예측한 값과 실제 값 간의 차이를 측정하여 모델이 잘 학습하고 있는 지에 대한 평가 기준을 제공한다. 여러 로스 함수들 중 MSE, MAE와 Huber loss 함수 중 선택하여 결정하였다.

2.4 Optimizer

최적화 함수는 Loss Function의 기울기를 사용해 가중치를 업데이트하여 손실 함수의 값을 최소화한다. Optimizer의 종류에는 Adam, NAdam, AdamW, RMSprop 등이 있으며 데이터 특성에 맞춰 최적화 방법을 선택한다.

2.5 Threshold

임계치 설정 기준에는 Accuracy, Precision, Recall, F1-Score가 있다. 이 중 F1-Score는 Precision 과 Recall의 조화 평균으로, 이상 탐지와 같은 문제에서

유용한 평가 지표로 작용한다.[4] F1-Score를 사용하면 모델이 비정상 데이터를 효과적으로 탐지하는 동시에 불필요한 오탐(False Positive)을 최소화하는데 유리하다. 따라서 모든 실험에서 F1-Score를 우선으로 설정하였다.

Ⅲ. 연구 결과

본 연구는 앞서 설명한 모델 분석 및 하이퍼파라미터를 수정하여 성능 평가 지표를 개선했다. F1-Score가 높은 것을 우선으로 하였고 F1-Score가 비슷할 경우 Recall 값이 좋은 것으로 선정하였다. 이는 비정상 데이터를 효과적으로 탐지하고 오탐(False Positive)을 줄이기 위함이다.

3.1 데이터 셋 및 환경

Eminist dataset을 사용하여 문자의 경우 이상, 숫자의 경우 정상으로 판단하였다. 데이터 셋은 0부터 255로 이루어져있다. 환경은 Google Colab T4 Gpu에서 하였다.

3.2 모델 구조 변경

3.2.1 레이어 수 및 히든 노드 수

[표 1]과 같이 레이어 수를 늘려가면서 성능을 비교하였다. 레이어를 늘리고 레이어 노드 수를 조정했다.784-680-500-320-160-70-32-70-160-320-500-680-784가 가장 좋은 성능지표를 보여주었다. 레이어가 갑자기 너무 많이 줄어들면 성능이 좋지 않았다.

3.3 Hyper parameter 결정

위에서 변경한 모델 구조 중 가장 좋은 모델 구조를 선택하여 Batch size는 [32, 64], Learning rate는 [0.0005, 0.001, 0.0015, 0.002]로 실험하였다. [표 2]와 같이 나왔고 Batch size는 64 Learning rate는 0.001이 가장 좋게 나왔다. 연구 과정에는 에폭에 대한 설명이 없지만 본 연구에 사용된 에폭 수는 모두 50이다.

3.4 활성화 함수 및 로스 함수 결정

위 실험에서 보여준 구조, Hyperparameter를 유지한 상태로 활성화 함수 및 로스 함수만 바꾸어 실험하였다.

3.5 최종 모델 구조

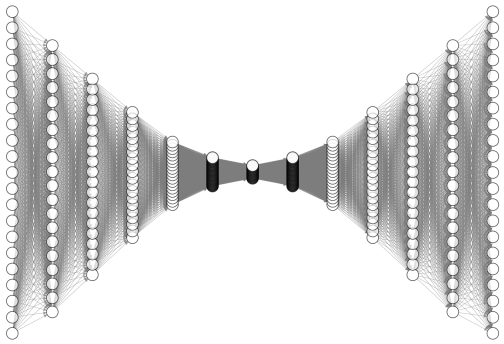

그림 2. 최종 오토인코더 모델 구조
Fig. 2. Final Autoencoder model architecture

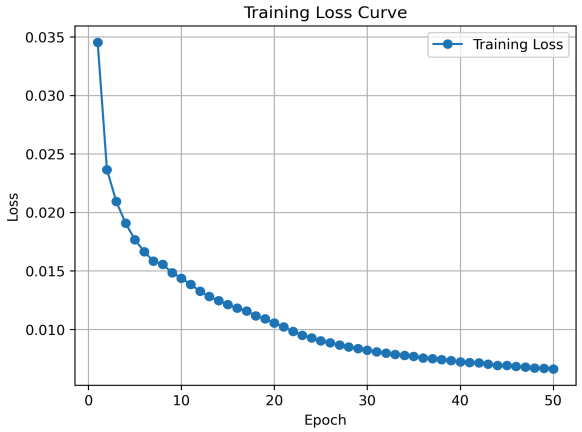

그림 3. 훈련 손실 그래프
Fig. 3. Training Loss Curve

표 1. 모델 구조 별 성능 지표
Table 1. Performance Metrics by Model Structure

	Model Structure	Accuracy	Precision	Recall	F1-score	layers
1	784-320-160-32-160-320-784	0.7584	0.7483	0.7789	0.7633	3
2	784-500-160-32-160-500-784	0.7864	0.7845	0.7897	0.7871	3
3	784-680-160-32-320-680-784	0.8049	0.8128	0.7921	0.8023	3
4	784-680-320-160-32-160-320-680-784	0.8650	0.8867	0.8370	0.8611	4
5	784-680-500-160-32-160-500-680-784	0.8587	0.8746	0.8375	0.8556	4
6	784-680-500-320-160-32-16-320-500-680-784	0.8643	0.9001	0.8196	0.8580	5
7	784-680-500-320-160-70-32-70-160-320-500-680-784	0.8741	0.9231	0.8163	0.8664	6
8	784-680-500-400-300-160-70-32-70-160-300-400-500-680-784	0.8514	0.8837	0.8092	0.8448	7

표 2. 하이퍼파라미터 별 성능 지표
Table 2. Performance Metrics by Hyperparameter

	Batch size	Learning rate	Accuracy	Precision	Recall	F1-score
	32	0.0001	0.8100	0.8189	0.7962	0.0.8074
1	32	0.0005	0.8614	0.9011	0.8119	0.8542
2	32	0.001	0.8651	0.9299	0.7898	0.8542
3	32	0.0015	0.8400	0.8737	0.7948	0.8324
4	32	0.002	0.8403	0.8781	0.7903	0.8319
	32	0.0025	0.8100	0.8035	0.8209	0.8121
	64	0.0001	0.7800	0.7656	0.8072	0.7858
5	64	0.0005	0.8611	0.9073	0.8043	0.8527
6	64	0.001	0.8741	0.9231	0.8163	0.8664
7	64	0.0015	0.8694	0.9148	0.8147	0.8618
8	64	0.002	0.8570	0.9033	0.7997	0.8484
	64	0.0025	0.8232	0.8204	0.8276	0.8240

표 3. 활성화 함수 및 로스 함수 성능 지표
Table 3. Performance Metrics of Activation Functions and Loss Functions

	Activation Function	Loss Function	Accuracy	Precision	Recall	F1-score
1	ReLU	MSE	0.8455	0.8971	0.7805	0.8347
2	LeakyReLU	MSE	0.8734	0.9192	0.8188	0.8664
3	ReLU	MAE	0.5467	0.5810	0.3347	0.4247
4	LeakyReLU	MAE	0.5565	0.5805	0.4075	0.4789
5	ReLU	Huber loss	0.8494	0.8921	0.7949	0.8407
6	LeakyReLU	Huber loss	0.8741	0.9231	0.8163	0.8664

Ⅳ. 결 론

본 연구에서는 오토인코더의 성능 최적화를 위해 Layers와 Hidden Nodes를 먼저 조정하여 모델의 구조적 변화를 시도하였고, 이 과정에서 다양한 조합을 실험하며 모델의 성능을 분석하였다. F1-Score가 높은 것을 우선으로 하였고 F1-Score가 비슷할 경우 Recall 값이 좋은 것으로 선정하였다. 이는 비정상 데이터를 효과적으로 탐지하고 오탐(False Positive)을 줄이기 위함이다.

[표 1]에서는 레이어 수를 늘릴 수록 대체적으로 성능이 올라갔지만 너무 많이 늘리면 성능이 줄어들었다. 히든 노드 수는 한 레이어를 지나갈 때마다 너무 많이 줄이지 않는 경우에 좋은 성능을 보였다. [표 2]에서는 Batch size와 Learning rate를 함께 튜닝하였다. Batch size는 64에서 더 좋은 결과가 나왔고 Learning rate는 0.001에서 더 좋은 결과 가 나왔다 Batch size가 128, 256 인 경우도 실험한 결과 시간이 오래걸리고 성능평가가 좋지 않다. 활성화 함수는 ReLu 보다 LeakyReLU가 더 좋은 성능을 보여줬다.[2] 로스 함수에서는 Huber loss가 MSE보다 미미하게 좋은 성능 지표를 보여주었다.

결론적으로 오토인코더의 모델 구조마다 데이터 처리 및 학습에 요구하는 특성이 다르기 때문에 모델 구조를 먼저 정하고 그 구조에 맞는 하이퍼파라미터들을 조정하는 방법이 오토인코더 튜닝을 위한 좋은 방법이라고 보여진다.

본 연구를 통해 비정상 데이터를 효과적으로 탐지할 수 있도록 오토인코더 하이퍼파라미터를 튜닝하였다. F1-Score, Recall, Precision 순으로 우선 순위를 두어 평가하였고 비정상 데이터를 정상을 판단하지 않는 것을 중점으로 튜닝하였다. 이는 오탐(False Positive)이 치명적인 산업계에 활용될 수 있다.

References

[1] James Bergstra, Rémi Bardenet, YoshuaBengio, Balázs Kégl, **Algorithms for hyper-parameter optimization**, Dec, 2011. (https://dl.acm.org/doi/10.5555/2986459.2986743)

[2] Bing Xu, Naiyan Wang, Tianqi Chen, Mu Li **Empirical Evaluation of Rectified Activations in Convolutional Network**, Nov,2015. (https://doi.org/10.48550/arXiv.1505.00853)

[3] Alex Krizhevsky, **One weird trick for parallelizing convolutional neural network**s, Apr, 2014. (https://doi.org/10.48550/arXiv.1404.5997)

[4] Z. Salekshahrezaee, J. L. Leevy and T. M. Khoshgoftaar, **"Feature Extraction for Class Imbalance Using a Convolutional Autoencoder and Data Sampling,"** *2021 IEEE 33rd International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 217-223, Washington, DC, USA, 2021. (https://doi.org/10.1109/ICTAI52525.2021.00037)